

SRO5



SYSTÈMES ET ALGORITHMES RÉPARTIS



**JEU DU
LOUP-
GAROU**

NOTRE GROUPE



Mathis DELMAERE



Eliott THOMANN



Elias LORGNIER



Eric BJARSTAL



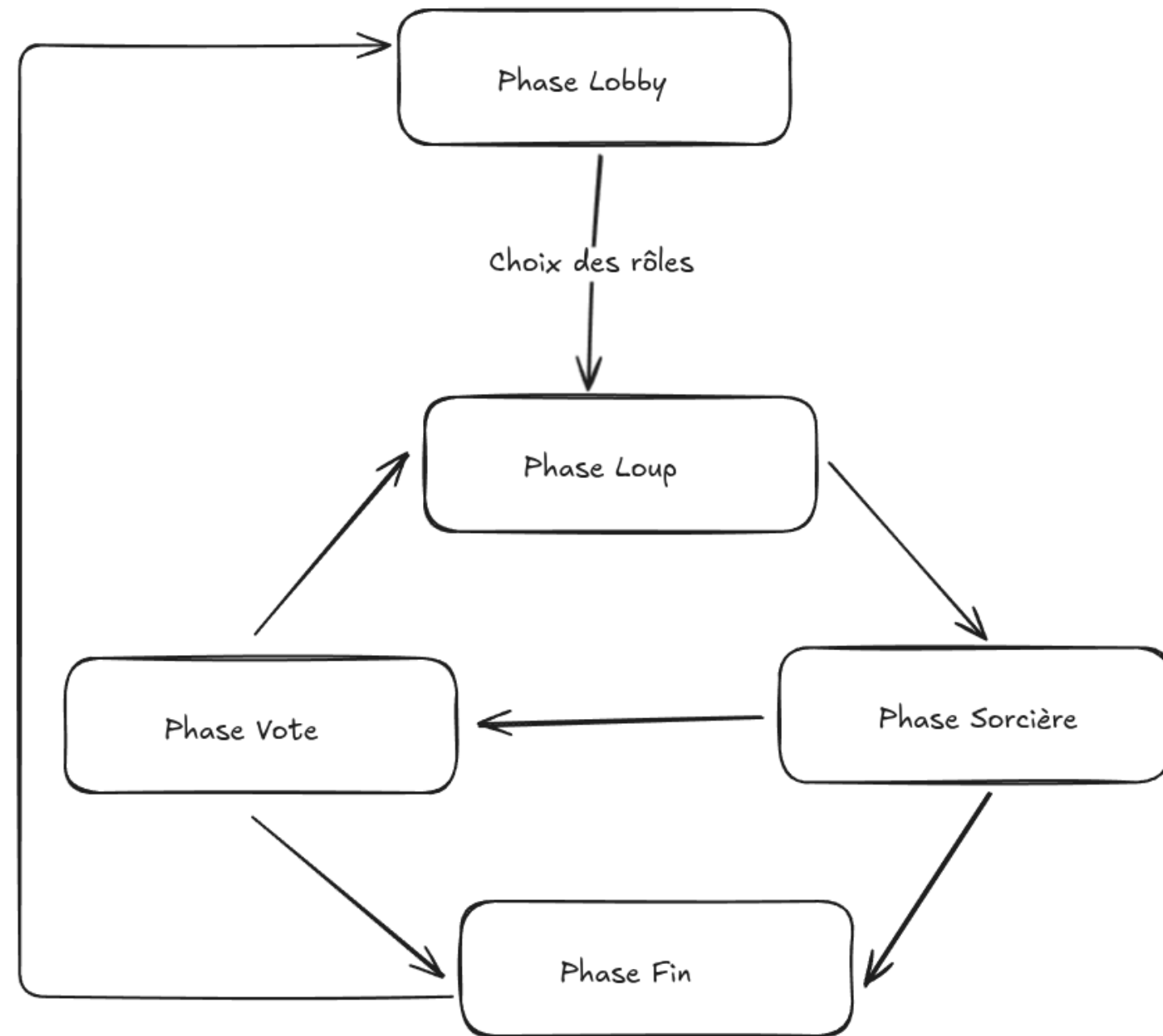
PLAN

1. Contexte et objectifs
2. Première architecture
3. Architecture établie
4. Le nouveau net
5. Adaptation du contrôle
6. Adaptation de l'application
7. Limites et perspectives
8. AI et répartition des tâches

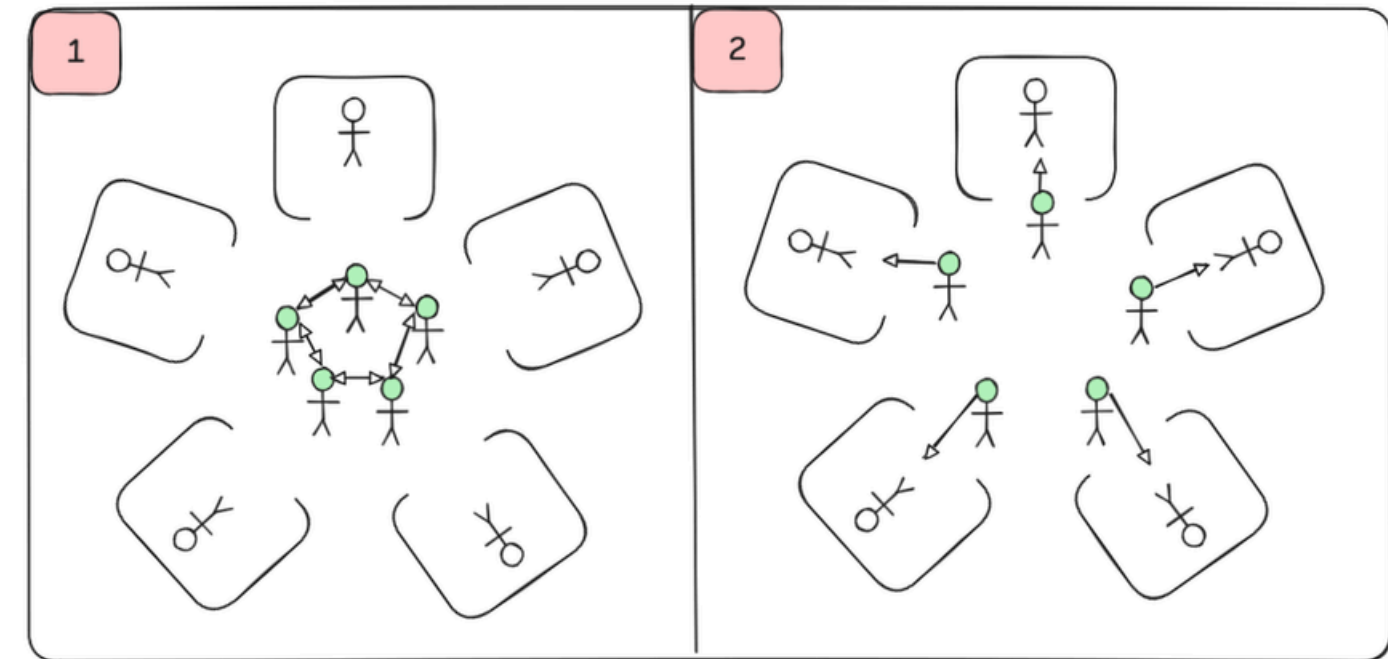


Adapter la première version de notre système réparti pour être capable de supporter l'arrivée de nouveaux joueurs, mais également le départ de participants, d'un point de vue réseau, composants d'un système distribué (horloge vectorielle, snapshots, etc.) et applicatif.

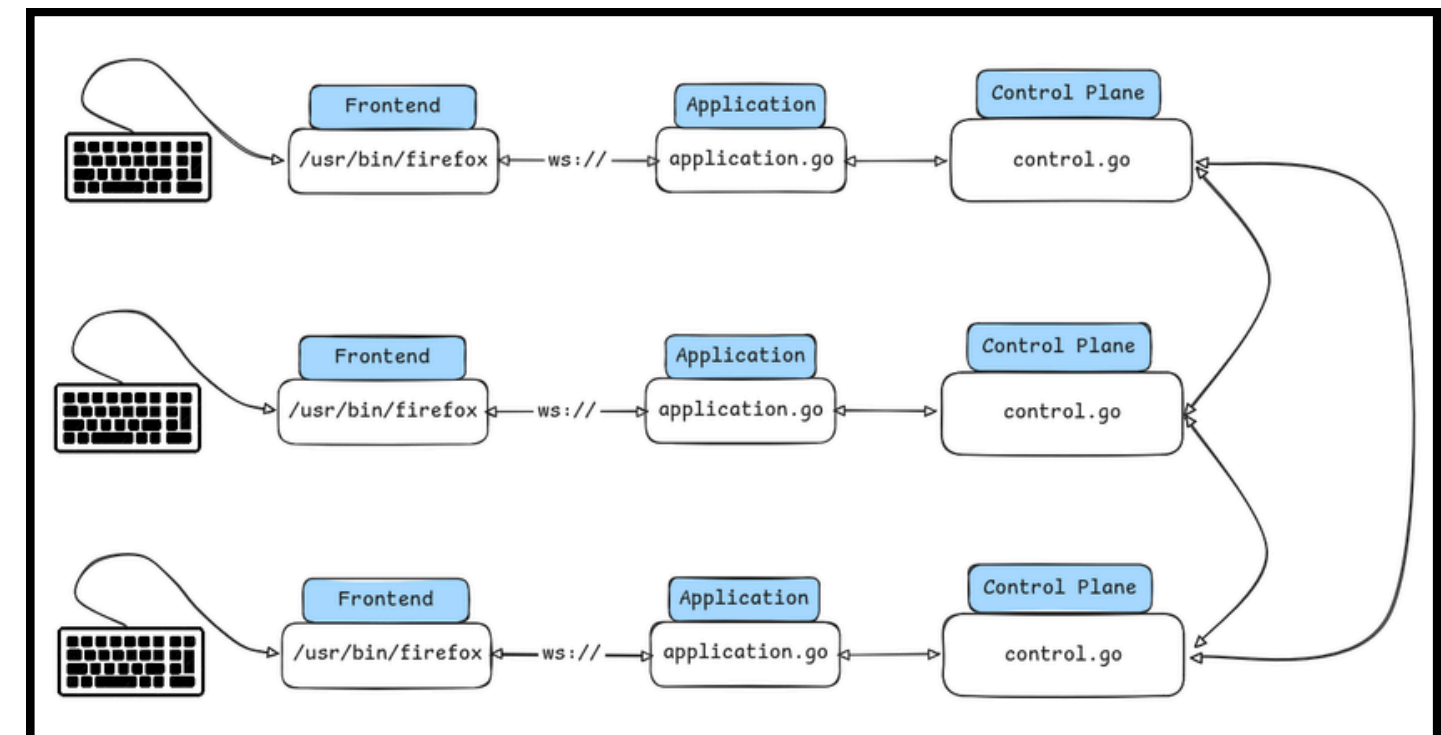
1. CONTEXTE ...



Enchainement des phases



Un Loup-garou distribué



Architecture du premier rendu

1. ... ET OBJECTIFS

Objectifs applicatifs :

Joueur ajouté :

- Il reçoit l'état du jeu
- Il est spectateur de la partie en cours
- Il joue à la partie suivante

Joueur supprimé :

- Le joueur est considéré comme mort
- La partie résout l'état actuel : fin d'une phase ? de la partie ?

Un peu d'UX :

- Si on ferme l'onglet, ou que la WebSocket se déconnecte, alors on quitte la partie

Objectifs d'architecture :

Décentralisation maximale:

- Pas besoin d'un site central ou d'un contrôleur spécifique au réseau
- Chaque site gère la connexion à son suivant et gère les connexions et déconnexions

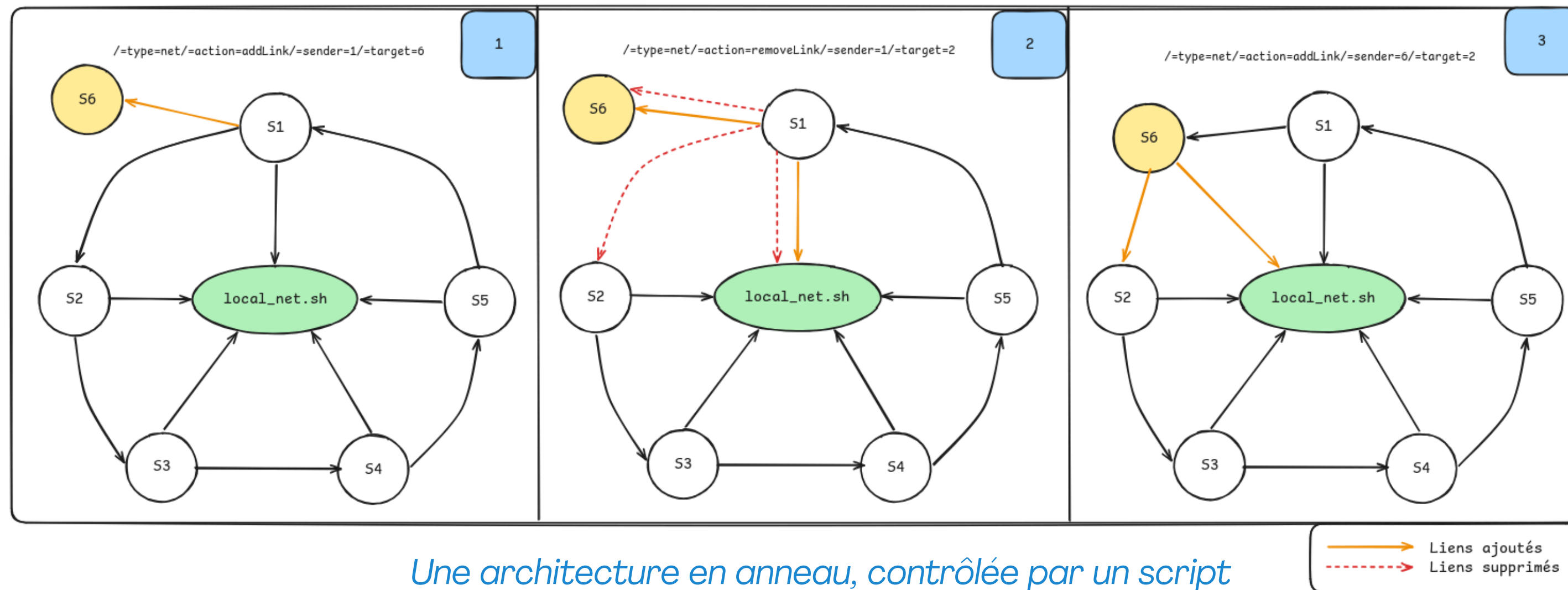
Gestion de la concurrence:

- Plusieurs sites peuvent partir ou s'ajouter simultanément sans problème
- Le tout sans perte de message

Séparation des couches:

- L'ajout au niveau contrôle et application est agnostique à la topologie du réseau
- Le réseau s'occupe seulement des branchements

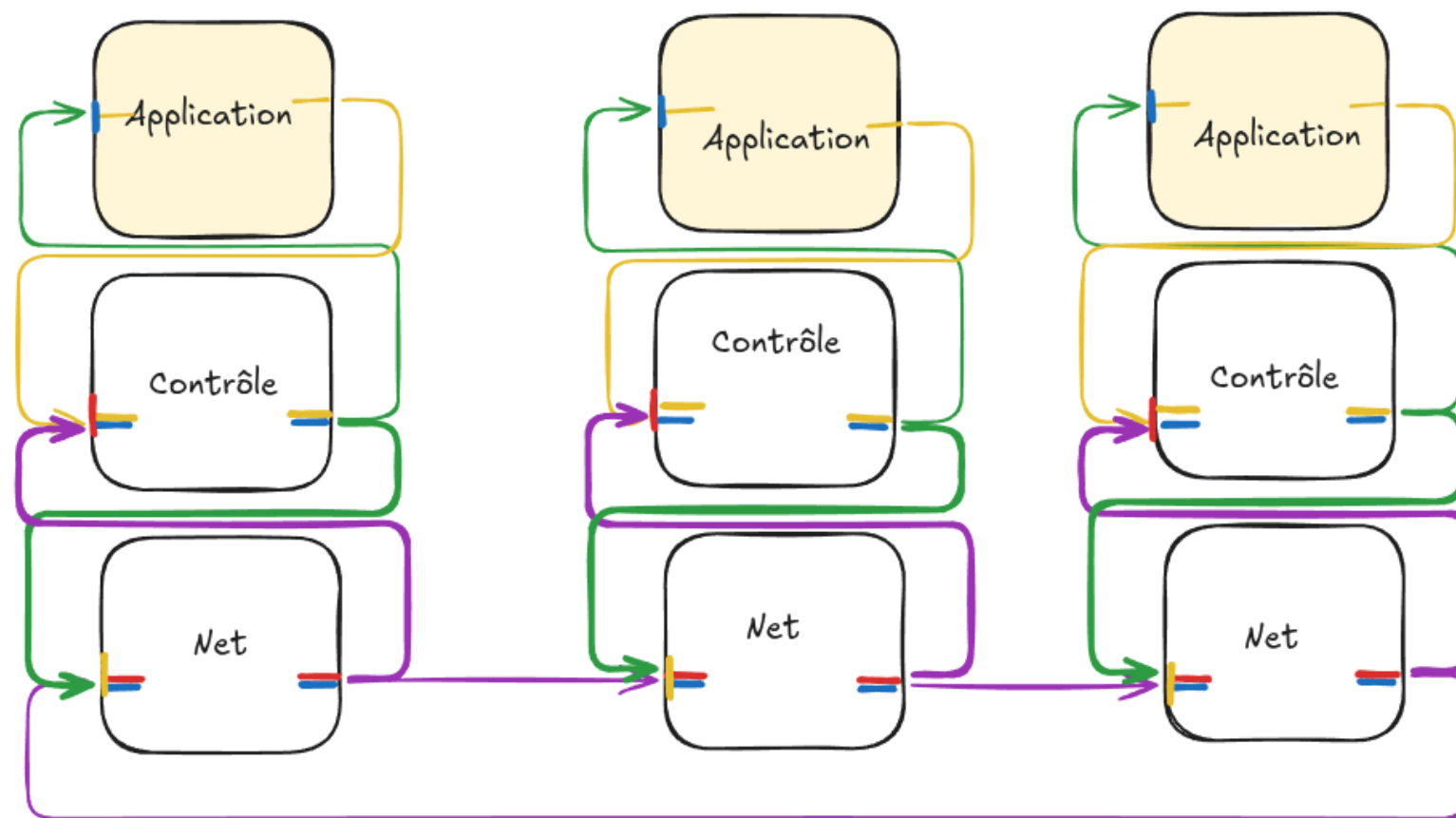
2. PREMIÈRE ARCHITECTURE



*Une architecture en anneau, contrôlée par un script
Shell*

3. ARCHITECTURE ETABLIE

- Message de type application
- Message de type contrôle
- Message de type net



Shéma des messages

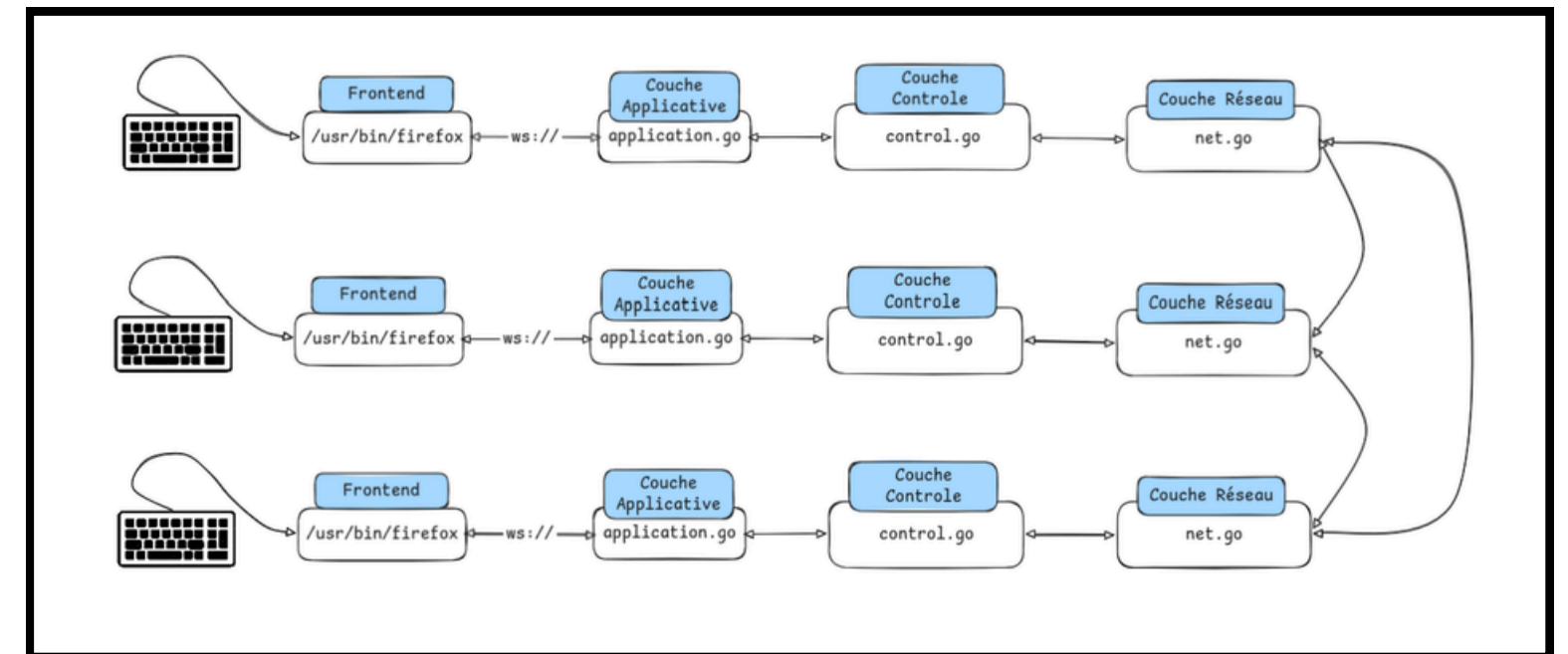


Schéma de l'architecture implémentée

4. LE NOUVEAU NET

AJOUT ET DÉPART

Processus d'ajout

Pour le nouveau site :

- Envoyer un message a un point de contact dans l'anneau
- Attendre d'être ajouté
- Diriger sa sortie vers son suivant quand ajouté

Pour l'anneau :

- Election d'un site qui ajoute le net (sur un critère variable)
 - On peut restreindre les participants en terme de degrés par noeud, latence réseau, etc.
- Gestion de la concurrence (une seule election à la fois)

Pour le “vainqueur” :

- Redirection de sa sortie vers le nouveau site
- Envoie de son ancien suivant au nouveau site
- Envoie “election terminée”

Processus de départ

Pour le nouveau site :

- Envoyer un message de départ en précisant son successeur dans l'anneau
- A la reception de son propre message : ``os.exit(0)``

Pour l'anneau :

- A la reception d'un message de départ venant de son successeur :
 - Transmettre le message jusqu'au site sortant
 - Rediriger sa sortie vers le site contenu dans le message de départ (théoriquement, le successeur de son successeur)

4. LE NOUVEAU NET

GESTION DE LA CONCURRENCE

Concurrence des elections

- Stocke les elections concurrentes
- L'election prioritaire et celle du site a l'identifiant le plus bas
- Une election ne peut pas être démarée si le site à déjà une election en cours
- Une election prioritaire "ecrase" l'election en cours
- Une election "moin prioritaire" est stockée en attente
- Les elections en attente sont relancées à la reception du message de fin d'election

Concurrence de l'élection et du départ

- Un site entrain de partir ne se présente pas a l'election
- Un site entrain de partir qui gagne une election la relance
- Un site qui part transmet ses elections en attente aux autres sites

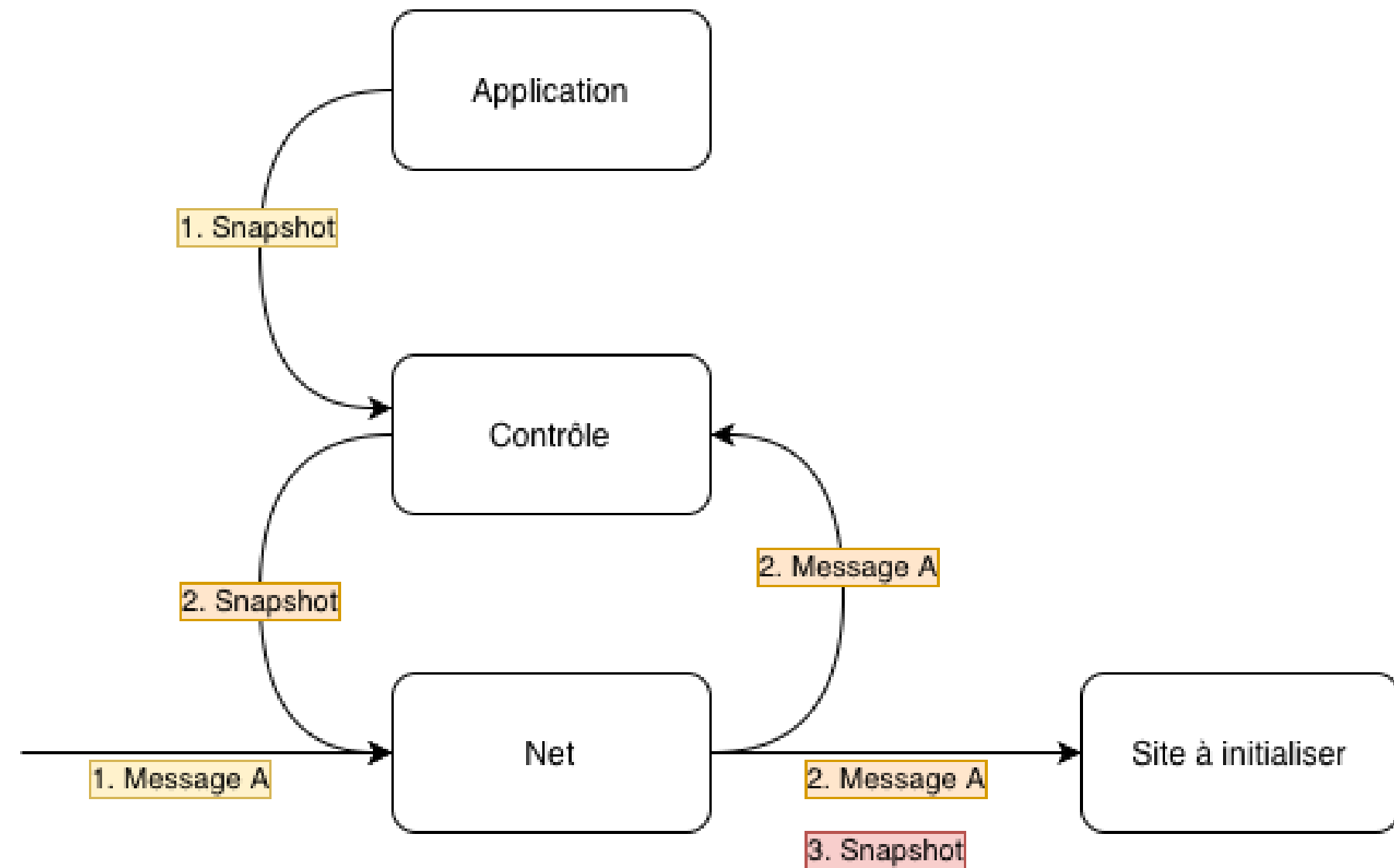
Concurrence des départs (deux sites consécutifs partent en même temps)

- Si je suis un site entrain de partir ET que je reçoit un message de départ de mon précédent, je modifie l'adresse contenue dans son message a l'adresse de mon suivant a moi.

5. ADAPTATION DU CONTRÔLE

Le cas problématique :

Initialisation incomplète due à la perte d'un message reçu avant l'initialisation sur le site local et après l'initialisation sur le site distant



5. ADAPTATION DU CONTRÔLE

Initialisation des sites :

- Deux modes de démarrage
- Boucle dédiée pour l'attente d'initialisation
- Tri des messages : tableau des estampilles des derniers messages reçus de chaque site

Sur un nouveau site

- Boucle d'attente d'initialisation
 - Message d'initialisation → sortie de la boucle
 - Autre messages → mise en queue
- Initialisation :
 - initialisation des variables de contrôle : tableau des sections critique, horloge vectorielle, vecteur des derniers messages reçus
 - lecture de la queue des messages, tri en fonction du tableau des estampilles

5.1 LES SNAPSHOTS

Algorithme 11 du cours :

- Coloriage rouge/blanc
- Préposts
- Terminaison explicite
- Daté par horloge vectorielle

Deux hypothèses du cours incompatibles avec un groupe qui évolue :

- Topologie en anneau
- N constant

Une arrivée ou un départ pendant la capture d'instantané empêche l'algorithme de se terminer ou bien résulte en une chimère qui ne respecte pas la causalité.

5.1 LES SNAPSHOTS

L'indépendance vis à vis de la topologie

Objectif : tout message de contrôle est livré exactement une fois à chaque autre membre, sans écho

Idée : remplacer l'anneau par une "entente" avec la couche **net**

```
// une émission = N-1 livraisons  
// (entente avec net, donc peu importe la topologie)
```

```
c.bilan += c.nbSites - 1
```


5.1 LES SNAPSHOTS

Robustesse au membership

Objectif : protéger les snapshots des changements de groupe

Idée : versionner les groupes en donne une **vue** à chaque site, elle est incrémentée à chaque ajout/retrait, et chaque message de contrôle porte la vue de son émetteur

Implémentation :

- Filtrage à la réception
- Changement de vue en une seule opération
- Une vue périmée ne relance pas une capture jetée

6. ADAPTATION DE L'APPLICATION

Joueur ajouté

- Le controller local nous transmet l'état du jeu de notre parrain
 - On s'approprie alors l'état du jeu,
 - On demande aux autres sites de nous considérer comme spectateur
- Côté frontend notre potentiel d'action et nul d'ici la prochaine partie

Un joueur quitte la partie

- Depuis un bouton “quitter” ou via fermeture de l'onglet (`window.removeEventListener('beforeunload', ...)`)
- On transmet la demande de quitter au controller, et on attend son retour pour pouvoir `os.exit(0)`
- Pour les autres sites
 - On résoud la phase actuelle : est-ce que la déconnexion met fin à une phase de vote ? ou même globalement au jeu ?
 - On demande au frontend d'afficher un toast pour notifier du départ d'un joueur
 - Puis on envoie le nouvel état au navigateur

7. LIMITES ET PERSPECTIVES

Perspectives

- Une architecture en anneau qui devrait pouvoir s'adapter à n'importe quelle autre topologie (connexe, sous couvert d'avoir une méthode pour marquer un message comme lu)
 - Demande de quitter l'approche Shell et de passer sur une approche Socket
- Déconnexion brutale : peut être géré en interceptant des erreurs `SIGPIPE`, puis en maintenant une liste d'au moins 2 successeurs dans le réseau
 - Ou plus facilement avec des communications bi-directionnels avec des heartbeat

Limites

- Caneaux FIFO
- Pas de perte de messages

8. RÉPARTITION ET IA

Mathis

- Première architecture
- Scripts
- Adapter la couche applicative et le frontend
- Debug sur l'arrivée d'un membre
- IA : Copilot en complétion de code

Eliott

- Définition en groupe de l'architecture et des algos
- Implémentation des adaptations sur la couche contrôle : support des départs et arrivées
- Pour les nouveaux sites : initialisation du contrôle
- IA : syntaxe go

Elias

- Création du net
- Pas d'IA

Eric

- Toute la partie snapshots
- Définition en groupe des attentes sur la communication control<->net
- Debug général
- IA: recherches sur le group membership, revues dans le code



MERCI !